

Guia Completo do Projeto SkyFlow

O **SkyFlow** é um sistema de gerenciamento de voos e reservas de passagens aéreas desenvolvido com foco na integração de bancos de dados relacionais (PostgreSQL) e não relacionais (MongoDB), utilizando o framework Spring Boot no backend e HTML/CSS/JavaScript no frontend.

Este guia tem como objetivo detalhar a configuração, a estrutura de pastas, o modelo de dados e o fluxo de execução ponta a ponta do projeto, servindo como um manual definitivo para novos desenvolvedores e usuários do sistema.

1. Configuração e Conexão com Bancos de Dados

O projeto exige a execução simultânea de dois bancos de dados: PostgreSQL para os dados transacionais e estruturados, e MongoDB para o armazenamento flexível de avaliações.

1.1. PostgreSQL e Flyway

O PostgreSQL é o banco de dados principal do sistema. Ele armazena informações sobre companhias aéreas, aeroportos, aeronaves, voos, passageiros e reservas.

Passos para configuração:

1. Instale o PostgreSQL em sua máquina.
2. Crie um banco de dados vazio chamado `skyflow_db`.
3. Configure as credenciais no arquivo `src/main/resources/application.properties`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/skyflow_db
spring.datasource.username=postgres
spring.datasource.password=senha123
```

Importante: Não é necessário criar as tabelas manualmente. O projeto utiliza o **Flyway**, uma ferramenta de migração de banco de dados. Ao iniciar a aplicação Spring Boot, o Flyway lerá os scripts localizados na pasta `src/main/resources/db/migration` e criará automaticamente todas as tabelas, views, procedures, triggers e inserirá os dados iniciais.

1.2. MongoDB

O MongoDB foi escolhido para armazenar as avaliações dos voos, pois comentários e notas possuem uma estrutura simples que pode crescer rapidamente sem a necessidade de

relacionamentos complexos.

Passos para configuração:

1. Instale o MongoDB e certifique-se de que o serviço está em execução na porta padrão (27017).
2. A conexão já está configurada no `application.properties`:

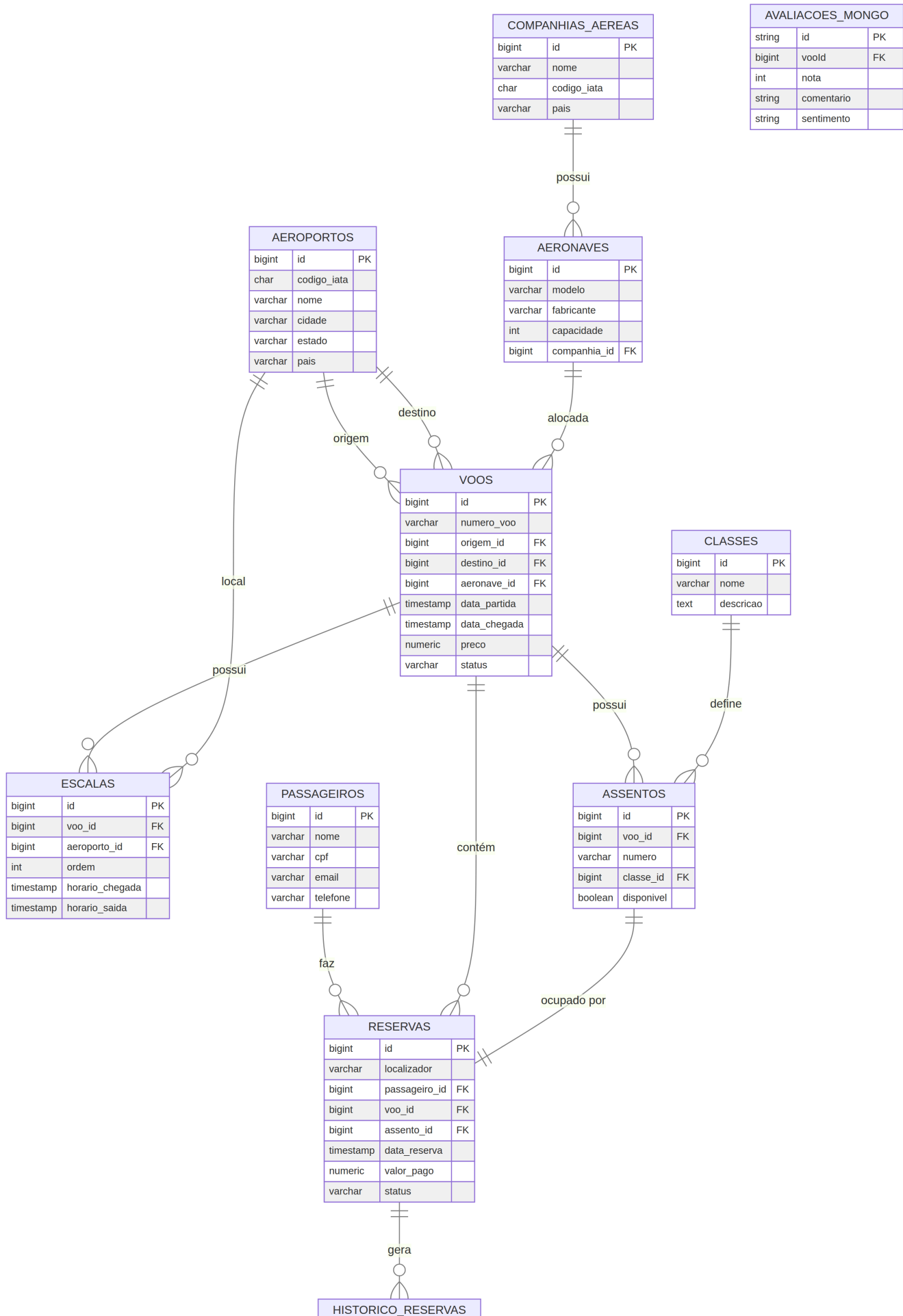
```
spring.data.mongodb.uri=mongodb://localhost:27017/skyflow_mongo
```

Criação da Coleção e Dados Iniciais: A coleção `avaliacoes` será criada automaticamente na primeira inserção. Para popular o banco com dados de exemplo, execute o script fornecido na pasta `mongo/`:

```
mongosh skyflow_mongo < mongo/insert_avaliacoes.js
```

2. Diagrama Entidade-Relacionamento (DER)

Abaixo está o diagrama que ilustra a estrutura do banco de dados relacional e a ligação lógica com o MongoDB.



bigint	id	PK
bigint	reserva_id	FK
varchar	operacao	
varchar	status_antigo	
varchar	status_novo	
timestamp	data_alteracao	

O relacionamento entre o PostgreSQL e o MongoDB ocorre de forma lógica através do campo `vooId` presente nos documentos do MongoDB, que faz referência à chave primária `id` da tabela `voos` no PostgreSQL.

3. Estrutura de Pastas e Arquitetura

O projeto segue uma arquitetura em camadas padrão do Spring Boot, separando responsabilidades de forma clara.

Diretório/Pasta	Descrição e Responsabilidade
<code>backup/</code>	Contém scripts Shell (<code>backup.sh</code> e <code>restore.sh</code>) para realizar o dump e a restauração do banco PostgreSQL.
<code>mongo/</code>	Scripts JavaScript para execução no <code>mongosh</code> , como inserção de dados iniciais (<code>insert_avaliacoes.js</code>) e exemplos de agregação.
<code>sql/</code>	Scripts SQL auxiliares contendo exemplos de consultas complexas, uso de <code>EXPLAIN ANALYZE</code> e transações.
<code>src/main/java/.../controller/</code>	Controladores REST que recebem as requisições HTTP do frontend e chamam os serviços adequados.
<code>src/main/java/.../service/</code>	Camada de regras de negócio. É aqui que ocorre a integração entre os dados do PostgreSQL e do MongoDB.
<code>src/main/java/.../repository/</code>	Interfaces JPA para comunicação com o PostgreSQL.
<code>src/main/java/.../mongo/repository/</code>	Interfaces Spring Data MongoDB para comunicação com o banco NoSQL.
<code>src/main/java/.../model/</code> e <code>mongo/model/</code>	Entidades que mapeiam as tabelas do PostgreSQL e os documentos do MongoDB, respectivamente.
<code>src/main/java/.../dto/</code>	Objetos de Transferência de Dados (Data Transfer Objects). Ex: <code>VooComAvaliacoesDTO</code> , que une dados dos dois bancos.
<code>src/main/resources/db/migration/</code>	Scripts do Flyway (<code>V1__...</code> até <code>V15__...</code>) executados sequencialmente para montar o banco relacional.

Diretório/Pasta	Descrição e Responsabilidade
<code>src/main/resources/static/</code>	Arquivos do frontend (HTML, CSS, JS). Contém as páginas <code>index.html</code> e <code>detalhes.html</code> .

4. Sprints de Execução do Banco de Dados

A construção do banco de dados relacional foi dividida em etapas lógicas, refletidas nos scripts do Flyway. Esta divisão simula “sprints” de desenvolvimento:

1. **Sprint 1 - Estrutura Base (V1):** Criação de todas as tabelas fundamentais (Companhias, Aeroportos, Aeronaves, Voos, Escalas, Classes, Assentos, Passageiros, Reservas e Histórico).
2. **Sprint 2 - Carga de Dados (V2 a V7):** Inserção de dados iniciais para popular o sistema e permitir testes.
3. **Sprint 3 - Otimização (V8):** Criação de índices nas tabelas mais acessadas (voos, reservas, passageiros) para melhorar a performance das consultas.
4. **Sprint 4 - Visualizações (V9 a V12):** Criação de Views e Materialized Views para simplificar relatórios complexos, como a listagem de assentos disponíveis e o consolidado de reservas.
5. **Sprint 5 - Lógica no Banco (V13 e V14):** Implementação de Procedures (ex: `reservar_assento()`) e Triggers (ex: registro automático no `historico_reservas`).
6. **Sprint 6 - Segurança (V15):** Criação de Roles (atendente, supervisor, cliente) e definição de permissões de acesso granulares.

5. Exemplo Ponta a Ponta: Integração Front e Back

O grande diferencial do SkyFlow é a forma como ele une dados de dois bancos diferentes e os entrega de forma transparente para o usuário. Abaixo, detalhamos esse fluxo usando a tela de detalhes do voo como exemplo.

5.1. O Frontend faz a requisição

No arquivo `src/main/resources/static/js/detalhes.js`, o JavaScript captura o ID do voo da URL e faz uma requisição para a API:

```
const params = new URLSearchParams(window.location.search);
const id = params.get("id");

fetch(`/voos/${id}/avaliacoes`)
  .then(response => response.json())
  .then(dados => {
    // Renderiza os dados do voo e as avaliações na tela
  });
```

5.2. O Controller recebe a chamada

O `VooController` intercepta a rota `/voos/{id}/avaliacoes` e repassa a responsabilidade para o `VooService`.

5.3. O Service une os dois mundos

No `VooService`, a mágica acontece. O serviço faz duas consultas distintas:

1. Busca os dados estruturados do voo no PostgreSQL usando o `VooRepository`.
2. Busca a lista de avaliações no MongoDB usando o `AvaliacaoRepository`.

Em seguida, ele empacota tudo no `VooComAvaliacoesDTO`:

```
Voo voo = vooService.buscarVoo(id); // PostgreSQL
List<Avaliacao> avaliacoes = vooService.buscarAvaliacoes(id); // MongoDB

VooComAvaliacoesDTO dto = new VooComAvaliacoesDTO();
dto.setVoo(voo);
dto.setAvaliacoes(avaliacoes);

return dto;
```

5.4. O Frontend exibe o resultado

O JSON retornado contém tanto as informações do voo (origem, destino, preço) quanto o array de avaliações (nota, comentário). O JavaScript então injeta esses dados dinamicamente no HTML, criando uma experiência fluida para o usuário, que não percebe que os dados vieram de tecnologias de banco de dados completamente diferentes.